

Expense Management API

API Specification

Version 1.0

Base URL: `http://localhost:8080`

Protocol: HTTPS (production) / HTTP (development)

Authentication: JWT Bearer Token

Content-Type: `application/json`

Total Endpoints: 48 (7 public + 41 authenticated)

Table of Contents

1. Overview
2. Authentication
3. Error Handling
4. Authentication Endpoints
5. Expense Endpoints
6. User Management Endpoints
7. Tenant Management Endpoints
8. Role Management Endpoints
9. Department Management Endpoints
10. Audit Log Endpoints
11. Enumerations

1. Overview

This document specifies the REST API endpoints for the Expense Management System. The API follows REST conventions with JSON request/response bodies.

1.1 Base URL

Environment	Base URL
Development	http://localhost:8080
Production	https://api.example.com

1.2 Response Envelope

All endpoints return responses wrapped in the following envelope:

Field	Type	Description
success	boolean	true if the request succeeded
message	string	Human-readable status message
data	T null	Response payload (null on errors)
timestamp	string	ISO-8601 timestamp

1.3 Pagination

List endpoints accept page, size, and sort query parameters. Responses include:

Field	Type	Description
content	array	Array of results
totalElements	integer	Total matching records
totalPages	integer	Total number of pages
number	integer	Current page (0-indexed)
size	integer	Page size

2. Authentication

The API uses **JWT-based stateless authentication**. Obtain a token via POST `/api/auth/login` and include it in the `Authorization: Bearer <token>` header.

2.1 Authentication Flow

Step	Description
1. Login	POST <code>/api/auth/login</code> with credentials. Returns access + refresh tokens (or MFA challenge).
2. MFA (if required)	POST <code>/api/auth/mfa/verify</code> with session token + 6-digit code. Returns full token pair.
3. Use API	Include Access Token in Authorization header. Token expires in 15 minutes.
4. Refresh	POST <code>/api/auth/refresh</code> (cookie-based). Returns new token pair (rotation).

2.2 Token Properties

Property	Value
Algorithm	HMAC-SHA512
Access Token Expiry	15 minutes
Refresh Token Expiry	7 days
Refresh Token Delivery	httpOnly, secure, SameSite=Strict cookie

2.3 Authorization

Access is controlled via method-level security annotations that check authority strings. Authority strings match the permission enum values. Multi-tenant scoping is enforced at the service layer.

3. Error Handling

Errors are returned as ApiResponse with success=false and a descriptive message.

3.1 HTTP Status Codes

Status	Meaning	When Returned
200 OK	Success	Request processed successfully
400 Bad Request	Validation Error	Invalid input, constraint violation, or business rule violation
401 Unauthorized	Authentication Failed	Invalid/missing credentials or token
403 Forbidden	Authorization Failed	Authenticated but lacks required permission
404 Not Found	Resource Not Found	Requested resource does not exist
409 Conflict	State Conflict	Invalid state transition (e.g., approving a rejected expense)
429 Too Many Requests	Rate Limited	Account locked or rate limit exceeded
500 Internal Server Error	Unexpected Error	Unhandled exception

3.2 Common Error Messages

Error Type	Status	Message
Validation failed	400	One or more fields failed validation
Invalid credentials	401	Username or password is incorrect
Account locked	429	Account locked due to too many failed attempts
Access denied	403	Insufficient permissions to perform this action
Resource not found	400	The requested resource does not exist
Invalid state transition	409	Cannot perform this action in the current state
Duplicate value	400	A record with the same value already exists

4. Authentication Endpoints

Base Path: /api/auth

Method	Path	Description	Auth
POST	/api/auth/login	Login with credentials	Public
POST	/api/auth/mfa/verify	Verify MFA code	Public
POST	/api/auth/refresh	Refresh access token (cookie)	Public
POST	/api/auth/forgot-password	Request password reset email	Public
POST	/api/auth/reset-password	Reset password with token	Public
POST	/api/auth/logout	Logout, revoke tokens	Authenticated
GET	/api/auth/me	Get current user profile	Authenticated
POST	/api/auth/change-password	Change password	Authenticated

POST /api/auth/login

Login with credentials

Request Body

Field	Type	Constraints	Description
usernameOrEmail	string	Required	Username or email address
password	string	Required, max 100 characters	Password
rememberMe	boolean	Optional	Remember-me flag

Response 200 — Success:

TokenResponse

Field	Type	Constraints	Description
accessToken	string	—	JWT access token
refreshToken	string	—	JWT refresh token (stripped in response)
tokenType	string	—	Always 'Bearer'
expiresIn	integer	—	Access token expiry in milliseconds
username	string	—	Authenticated username
roles	array of strings	—	Granted authorities

Response 200 — MFA Required:

MfaLoginResponse

Field	Type	Constraints	Description
mfaRequired	boolean	—	Always true
mfaSessionToken	string	—	Temporary token for MFA verification
method	string	—	MFA method (TOTP, EMAIL)
expiresIn	integer	—	Session token expiry in milliseconds

Status Codes:

Code	Description
200	Login successful (tokens returned or MFA challenge)
400	Validation failed
401	Invalid credentials
429	Account locked due to too many failed attempts

POST /api/auth/mfa/verify

Verify MFA code during login

Request Body

Field	Type	Constraints	Description
mfaSessionToken	string	Required	MFA session token from login response
code	string	Required, exactly 6 characters	6-digit MFA code

Response 200 — Success:

TokenResponse

Field	Type	Constraints	Description
accessToken	string	—	JWT access token
refreshToken	string	—	JWT refresh token
tokenType	string	—	Always 'Bearer'
expiresIn	integer	—	Access token expiry in milliseconds
username	string	—	Authenticated username
roles	array of strings	—	Granted authorities

Status Codes:

Code	Description
200	MFA verification successful
400	Invalid or expired MFA session token
401	Invalid MFA code

POST /api/auth/refresh

Refresh access token

Reads refresh token from httpOnly cookie. Returns new access + refresh token pair (rotation). Old refresh token is revoked.

Response 200 — Success:

TokenResponse

Field	Type	Constraints	Description
accessToken	string	—	JWT access token
refreshToken	string	—	JWT refresh token (stripped in response)
tokenType	string	—	Always 'Bearer'
expiresIn	integer	—	Access token expiry in milliseconds
username	string	—	Authenticated username
roles	array of strings	—	Granted authorities

Status Codes:

Code	Description
200	Token refreshed successfully
401	Invalid or expired refresh token

POST /api/auth/forgot-password

Request password reset email

Request Body

Field	Type	Constraints	Description
email	string	Required, valid email format	Email address for reset link

Status Codes:

Code	Description
200	Reset email sent (always returns success for security)
400	Validation failed

POST /api/auth/reset-password

Reset password with token

Request Body

Field	Type	Constraints	Description
token	string	Required	Password reset token from email
newPassword	string	Required, custom password rules	New password
confirmPassword	string	Required	Confirmation of new password

Status Codes:

Code	Description
200	Password reset successful
400	Invalid/expired token, passwords do not match, validation failed

POST /api/auth/logout

Logout, revoke tokens

Blacklists the access token JTI in Redis. Revokes refresh tokens. Clears the refresh token cookie.

Status Codes:

Code	Description
200	Logout successful

GET /api/auth/me

Get current user profile

Response 200 — Success:

UserProfileResponse

Field	Type	Constraints	Description
username	string	—	Username
email	string	—	Email address
firstName	string	—	First name
lastName	string	—	Last name
roles	array of strings	—	Assigned role names
enabled	boolean	—	Whether account is enabled
mfaEnabled	boolean	—	Whether MFA is enabled
mfaMethod	string	—	MFA method if enabled

POST /api/auth/change-password

Change password

Request Body

Field	Type	Constraints	Description
currentPassword	string	Required	Current password
newPassword	string	Required, custom password rules	New password
confirmPassword	string	Required	Confirmation of new password

Status Codes:

Code	Description
200	Password changed successfully
400	Current password incorrect, validation failed

5. Expense Endpoints

Base Path: /api/expenses

Method	Path	Description	Auth
GET	/api/expenses	List expenses (paginated, filtered)	EXPENSE_READ
GET	/api/expenses/{id}	Get expense by ID	EXPENSE_READ
POST	/api/expenses	Create expense	EXPENSE_CREATE
PUT	/api/expenses/{id}	Update expense	EXPENSE_UPDATE
POST	/api/expenses/{id}/cancel	Cancel expense	EXPENSE_UPDATE
POST	/api/expenses/{id}/approve	Approve expense	EXPENSE_APPROVE
POST	/api/expenses/{id}/reject	Reject expense	EXPENSE_REJECT
POST	/api/expenses/{id}/process	Process for payment	EXPENSE_PROCESS

GET /api/expenses

List expenses — Scope depends on the user's role.

Query Parameters:

Parameter	Type	Required	Description
status	string (enum)	No	Filter by status
tenantId	integer	No	Filter by tenant ID
departmentId	integer	No	Filter by department ID
page	integer	No	Page number (default 0)
size	integer	No	Page size (default 20)
sort	string	No	Sort field,direction (e.g. submissionDate,desc)

Response 200 — Success:

Returns a paginated list of expense objects.

Status Codes:

Code	Description
200	Expenses listed successfully
400	Invalid filter parameters

POST /api/expenses

Create expense

Request Body

Field	Type	Constraints	Description
title	string	Required, max 200 characters	Expense title
description	string	Optional, max 1000 characters	Optional description
amount	decimal	Required, positive, up to 12 digits with 4 decimal places	Expense amount
category	string	Required, max 50 characters	Expense category
departmentId	integer	Optional	Optional department association

Status Codes:

Code	Description
200	Expense created successfully
400	Validation failed

ExpenseResponse (used by all expense endpoints):

ExpenseResponse

Field	Type	Constraints	Description
id	integer	—	Expense ID
title	string	—	Title
description	string	—	Description
amount	decimal	—	Amount
category	string	—	Category
status	string (enum)	—	PENDING APPROVED REJECTED CANCELLED PROCESSED
submissionDate	timestamp	—	When submitted
decisionDate	timestamp	—	When approved/rejected
processedDate	timestamp	—	When processed for payment
ownerId	integer	—	Owner user ID
ownerUsername	string	—	Owner username
departmentId	integer	—	Department ID
departmentName	string	—	Department name
tenantId	integer	—	Tenant ID
tenantName	string	—	Tenant name
approvedById	integer	—	Approver user ID
approvedByUsername	string	—	Approver username
rejectedById	integer	—	Rejector user ID
rejectedByUsername	string	—	Rejector username
processedById	integer	—	Processor user ID
processedByUsername	string	—	Processor username
updatedAt	timestamp	—	Last update timestamp

PUT /api/expenses/{id}

Update expense — Only the owner can update a PENDING expense.

Request Body

Field	Type	Constraints	Description
title	string	Required, max 200 characters	Expense title
description	string	Optional, max 1000 characters	Optional description
amount	decimal	Required, positive, up to 12 digits with 4 decimal places	Expense amount
category	string	Required, max 50 characters	Expense category

Status Codes:

Code	Description
200	Expense updated successfully
400	Validation failed or expense not in PENDING status
403	Not the expense owner

POST /api/expenses/{id}/cancel

Owner cancels a PENDING expense

Status Codes:

Code	Description
200	Success
400	Not in PENDING status
403	Not the owner

POST /api/expenses/{id}/approve

Manager approves a PENDING expense

Status Codes:

Code	Description
200	Success
400	Not in PENDING status
403	Not a manager

POST /api/expenses/{id}/reject

Manager rejects a PENDING expense

Status Codes:

Code	Description
200	Success
400	Not in PENDING status
403	Not a manager

POST /api/expenses/{id}/process

Finance processes an APPROVED expense

Status Codes:

Code	Description
200	Success
400	Not in APPROVED status
403	Not finance

6. User Management Endpoints

Base Path: /api/management/users

Method	Path	Description	Auth
POST	/api/management/users	Create user	USER_CREATE
GET	/api/management/users	List users (paginated)	USER_READ
GET	/api/management/users/{id}	Get user by ID	USER_READ
PUT	/api/management/users/{id}	Update user	USER_WRITE
DELETE	/api/management/users/{id}	Delete user	USER_DELETE
POST	/api/management/users/{id}/enable	Toggle enabled state	USER_ENABLE
POST	/api/management/users/{id}/roles	Assign role to user	USER_ASSIGN_ROLE
DELETE	/api/management/users/{id}/roles	Remove role from user	USER_ASSIGN_ROLE
POST	/api/management/users/{id}/mfa/enable	Enable MFA for user	USER_WRITE
POST	/api/management/users/{id}/mfa/disable	Disable MFA for user	USER_WRITE
POST	/api/management/users/{id}/mfa/reset	Reset MFA for user	USER_WRITE

UserResponse (used by all user endpoints):

UserResponse

Field	Type	Constraints	Description
id	integer	—	User ID
username	string	—	Username
email	string	—	Email
firstName	string	—	First name
lastName	string	—	Last name
enabled	boolean	—	Account enabled
accountNonLocked	boolean	—	Account locked status
departmentId	integer	—	Department ID
departmentName	string	—	Department name
roles	array of strings	—	Assigned role names
permissions	array of strings	—	All effective permissions
mfaEnabled	boolean	—	MFA enabled
mfaMethod	string	—	MFA method
createdAt	timestamp	—	Creation timestamp
updatedAt	timestamp	—	Last update timestamp

POST /api/management/users

Create a new user

Request Body

Field	Type	Constraints	Description
username	string	Required, 3–50 characters	Username
email	string	Required, valid email, max 100 characters	Email
password	string	Required, custom password rules	Password
firstName	string	Optional, max 50 characters	First name
lastName	string	Optional, max 50 characters	Last name
roleName	string	Optional, max 50 characters	Optional role name to assign
tenantId	integer	Optional	Optional tenant ID
departmentId	integer	Required	Required department ID

Status Codes:

Code	Description
200	User created successfully
400	Username/email already exists, validation failed

PUT /api/management/users/{id}

Update an existing user

Request Body

Field	Type	Constraints	Description
firstName	string	Optional, max 50 characters	First name
lastName	string	Optional, max 50 characters	Last name
departmentId	integer	Optional	Optional department ID

Status Codes:

Code	Description
200	User updated successfully
400	Validation failed

DELETE /api/management/users/{id}

Delete a user

Status Codes:

Code	Description
200	User deleted successfully
400	Cannot delete the last admin in a tenant

POST /api/management/users/{id}/enable

Enable or disable a user account

Request Body

Field	Type	Constraints	Description
enabled	boolean	Required	Desired enabled state

Status Codes:

Code	Description
200	User enabled state updated
400	Cannot disable the last admin in a tenant

POST /api/management/users/{id}/roles

Assign a role to a user

Request Body

Field	Type	Constraints	Description
roleName	string	Required	Role name to assign

Status Codes:

Code	Description
200	Role assigned successfully
400	Role not found, validation failed

DELETE /api/management/users/{id}/roles

Remove a role from a user

Request Body

Field	Type	Constraints	Description
roleName	string	Required	Role name to remove

Status Codes:

Code	Description
200	Role removed successfully
400	Role not found

POST /api/management/users/{id}/mfa/enable

Enable MFA for a user — Returns the MFA secret and QR code URI for TOTP setup.

Request Body

Field	Type	Constraints	Description
method	string (enum)	Required	MFA method (TOTP or EMAIL)

Response 200 — Success:

MfaSetupResponse

Field	Type	Constraints	Description
qrUri	string	—	OTP auth URI for QR code generation (TOTP only, null for EMAIL)
secret	string	—	MFA secret key (TOTP only, null for EMAIL)
method	string (enum)	—	Configured MFA method

Status Codes:

Code	Description
200	MFA enabled successfully
400	MFA is already enabled for this user
403	Insufficient permissions

POST /api/management/users/{id}/mfa/disable

Disable MFA for a user

Status Codes:

Code	Description
200	MFA disabled successfully
400	MFA is not enabled for this user
403	Insufficient permissions

POST /api/management/users/{id}/mfa/reset

Reset MFA for a user — Invalidates the old secret and generates a new one.

Request Body

Field	Type	Constraints	Description
method	string (enum)	Required	MFA method (TOTP or EMAIL)

Status Codes:

Code	Description
200	MFA reset successfully
400	MFA is not enabled for this user
403	Insufficient permissions

7. Tenant Management Endpoints

Base Path: /api/management/tenants

Method	Path	Description	Auth
GET	/api/management/tenants	List tenants (paginated, filtered)	TENANT_READ
GET	/api/management/tenants/{id}	Get tenant by ID	TENANT_READ
POST	/api/management/tenants	Create tenant	TENANT_CREATE
PUT	/api/management/tenants/{id}	Update tenant	TENANT_UPDATE
DELETE	/api/management/tenants/{id}	Delete tenant	TENANT_DELETE

TenantResponse (used by all tenant endpoints):

TenantResponse

Field	Type	Constraints	Description
id	integer	—	Tenant ID
name	string	—	Tenant name
status	string (enum)	—	ACTIVE INACTIVE SUSPENDED
createdAt	timestamp	—	Creation timestamp

POST /api/management/tenants

Create a new tenant

Request Body

Field	Type	Constraints	Description
name	string	Required, max 100 characters	Tenant name
status	string (enum)	Required	ACTIVE, INACTIVE, or SUSPENDED

Status Codes:

Code	Description
200	Tenant created successfully
400	Tenant name already exists, validation failed

PUT /api/management/tenants/{id}

Update an existing tenant

Request Body

Field	Type	Constraints	Description
name	string	Required, max 100 characters	Tenant name
status	string (enum)	Required	ACTIVE, INACTIVE, or SUSPENDED

Status Codes:

Code	Description
200	Tenant updated successfully
400	Validation failed

DELETE /api/management/tenants/{id}

Delete a tenant

Status Codes:

Code	Description
200	Tenant deleted successfully
400	Tenant has active users or departments

8. Role Management Endpoints

Base Path: `/api/management/roles`

Method	Path	Description	Auth
GET	<code>/api/management/roles</code>	List roles (paginated)	ROLE_READ
GET	<code>/api/management/roles/{id}</code>	Get role by ID	ROLE_READ
POST	<code>/api/management/roles</code>	Create role	PLATFORM_ADMIN
PUT	<code>/api/management/roles/{id}</code>	Update role	PLATFORM_ADMIN
DELETE	<code>/api/management/roles/{id}</code>	Delete role	PLATFORM_ADMIN
POST	<code>/api/management/roles/{id}/permissions</code>	Add permission	PLATFORM_ADMIN
DELETE	<code>/api/management/roles/{id}/permissions</code>	Remove permission	PLATFORM_ADMIN

RoleResponse (used by all role endpoints):

RoleResponse

Field	Type	Constraints	Description
<code>id</code>	integer	—	Role ID
<code>name</code>	string	—	Role name
<code>title</code>	string	—	Human-readable title
<code>description</code>	string	—	Description
<code>permissions</code>	array of strings	—	Assigned permissions

POST `/api/management/roles`

Create a new role

Request Body

Field	Type	Constraints	Description
<code>name</code>	string	Required, max 50 characters	Role name (unique key)
<code>title</code>	string	Optional, max 100 characters	Human-readable title
<code>description</code>	string	Optional, max 255 characters	Description

Status Codes:

Code	Description
200	Role created successfully
400	Role name already exists, validation failed

POST /api/management/roles/{id}/permissions

Add a permission to a role

Request Body

Field	Type	Constraints	Description
permission	string (enum)	Required	Permission enum value

Status Codes:

Code	Description
200	Permission added successfully
400	Permission already assigned, validation failed

DELETE /api/management/roles/{id}/permissions

Remove a permission from a role

Request Body

Field	Type	Constraints	Description
permission	string (enum)	Required	Permission enum value

Status Codes:

Code	Description
200	Permission removed successfully
400	Permission not found on role

9. Department Management Endpoints

Base Path: /api/management/departments

Method	Path	Description	Auth
GET	/api/management/departments	List departments (paginated)	DEPARTMENT_READ
GET	/api/management/departments/{id}	Get department by ID	DEPARTMENT_READ
POST	/api/management/departments	Create department	DEPARTMENT_CREATE
PUT	/api/management/departments/{id}	Update department	DEPARTMENT_UPDATE
DELETE	/api/management/departments/{id}	Delete department	DEPARTMENT_DELETE

DepartmentResponse (used by all department endpoints):

DepartmentResponse

Field	Type	Constraints	Description
id	integer	—	Department ID
name	string	—	Department name
tenantId	integer	—	Parent tenant ID
tenantName	string	—	Parent tenant name
managerIds	array of integers	—	Manager user IDs
managerUsernames	array of strings	—	Manager usernames

POST /api/management/departments

Create a new department

Request Body

Field	Type	Constraints	Description
name	string	Required, max 100 characters	Department name
tenantId	integer	Required	Parent tenant ID
managerIds	array of integers	Optional	Optional list of manager user IDs

Status Codes:

Code	Description
200	Department created successfully
400	Department name already exists in tenant, validation failed

PUT /api/management/departments/{id}**Update an existing department****Request Body**

Field	Type	Constraints	Description
name	string	Required, max 100 characters	Department name
managerIds	array of integers	Optional	Optional list of manager user IDs

Status Codes:

Code	Description
200	Department updated successfully
400	Validation failed

DELETE /api/management/departments/{id}**Delete a department****Status Codes:**

Code	Description
200	Department deleted successfully
400	Department has active users or expenses

10. Audit Log Endpoints

Base Path: /api/management/audit

Method	Path	Description	Auth
GET	/api/management/audit	List audit logs (paginated)	AUDIT_LOG_READ
GET	/api/management/audit/{id}	Get audit log by ID	AUDIT_LOG_READ

AuditLogResponse

AuditLogResponse

Field	Type	Constraints	Description
id	integer	—	Audit log ID
actorId	integer	—	Actor user ID
actorUsername	string	—	Actor username
tenantId	integer	—	Tenant ID
action	string	—	Action performed
resourceType	string	—	Resource type (USER, EXPENSE, TENANT)
resourceId	string	—	Resource ID
details	string	—	Additional details
timestamp	timestamp	—	When the action occurred

GET /api/management/audit

List audit logs — Super admins see all logs; tenant admins see their tenant's logs.

Query Parameters:

Parameter	Type	Required	Description
page	integer	No	Page number (default 0)
size	integer	No	Page size (default 20)
sort	string	No	Sort field,direction

11. Enumerations

11.1 ExpenseStatus

Value	Description
PENDING	Expense submitted, awaiting review
APPROVED	Expense approved by a manager
REJECTED	Expense rejected by a manager
CANCELLED	Expense cancelled by the owner
PROCESSED	Approved expense processed for payment

11.2 TenantStatus

Value	Description
ACTIVE	Tenant is active and operational
INACTIVE	Tenant is temporarily inactive
SUSPENDED	Tenant is suspended

11.3 MfaMethod

Value	Description
NONE	No MFA configured
TOTP	Time-based One-Time Password (authenticator app)
EMAIL	Email-based MFA codes

11.4 UserPermission

Category	Permissions
Tenant	TENANT_READ, TENANT_CREATE, TENANT_UPDATE, TENANT_DELETE
User	USER_READ, USER_WRITE, USER_CREATE, USER_UPDATE, USER_DELETE, USER_ENABLE, USER_ASSIGN_ROLE
Role	ROLE_READ, ROLE_WRITE, ROLE_DELETE, ROLE_ASSIGN_PERMISSION
Department	DEPARTMENT_READ, DEPARTMENT_CREATE, DEPARTMENT_UPDATE, DEPARTMENT_DELETE
Expense	EXPENSE_READ, EXPENSE_READ_ALL, EXPENSE_CREATE, EXPENSE_UPDATE, EXPENSE_DELETE, EXPENSE_APPROVE, EXPENSE_REJECT, EXPENSE_PROCESS
MFA	MFA_MANAGE
Reporting & Audit	REPORT_READ, AUDIT_LOG_READ